

The Learning-Rate Finder, Worked Out by Hand

One exponential sweep to read a good learning rate off the loss curve

A complete numerical walk-through of the *learning-rate range test*: instead of grid-searching the single most important hyper-parameter, you run *one* short training pass while multiplying the learning rate by a constant factor every mini-batch, so it climbs exponentially from $\text{lr}_{\min}$ to $\text{lr}_{\max}$. You then read a good learning rate straight off the resulting loss-vs-lr curve. Every number below — the multiplier q , the learning rate at each step, the loss, the slope, and the suggested learning rate — was produced by the reference program in Appendix A, so the arithmetic is exact and self-consistent.

What this document does. It fixes a concrete sweep, $\text{lr}_{\min} = 10^{-5}$, $\text{lr}_{\max} = 1$, over $\text{num_iter} = 100$ mini-batches, derives the per-step multiplier $q = (\text{lr}_{\max}/\text{lr}_{\min})^{1/\text{num_iter}}$, tabulates the learning rate at $i = 0, 20, 50, 80, 100$, attaches a plausible loss to each, and then locates the suggested learning rate two ways — the point of *steepest descent* and the *minimum-loss-over-ten* rule — showing they agree.

How to read this. Each idea gets its own worked step with exact numbers, plus coloured boxes that isolate the intuition, the situation where the test is the right tool, and the two classic traps. Read this to understand *why* a single geometric sweep is enough, and *which* point on the curve to pick.

Concepts covered, in order: the geometric (exponential) learning-rate schedule · deriving the multiplier q and "steps per decade" · the three regimes of the loss curve (flat, descent, divergence) · the index→learning-rate map · the slope $d\text{loss}/d\log_{10}\text{lr}$ and the steepest-descent pick · the minimum-loss-over-ten rule · the $4\times$ divergence guard · resetting the perturbed weights.

1 The setup: one cheap sweep instead of a grid search

The learning rate is the single most consequential knob in training: too small and the loss barely moves; too large and it oscillates or blows up to NaN. Grid-searching it means a full training run *per* candidate. The range test replaces that with *one* short run. Over a few hundred mini-batches you take a normal optimiser step each batch, but before the next batch you *multiply* the learning rate by a fixed factor $q > 1$, so it sweeps a wide range on a log scale.

Quantity	Symbol	Value here
Smallest learning rate	$\text{lr}_{\min}$	10^{-5}
Largest learning rate	$\text{lr}_{\max}$	1
Number of sweep steps	num_iter	100
Per-step multiplier	q	1.122
Schedule	—	geometric: $\text{lr}_i = \text{lr}_{\min} q^i$
Loss recorded per step	ℓ_i	one mini-batch loss

The example. A small 3-class classifier, cross-entropy loss. At initialisation the loss sits near $\ln 3 \approx 1.099$ (uniform guessing). As the swept learning rate enters its useful band the loss falls steeply; once the learning rate is too large the updates overshoot and the loss climbs back up and diverges. We will read the good learning rate off exactly that shape.

Intuition

Each step of the sweep is a tiny, almost-free probe: "if I trained with *this* learning rate for one batch, which way does the loss move?" Because the model only changes a little between adjacent steps, the recorded curve is essentially a single picture of *loss as a function of learning rate*. The steepest-descent learning rate is the one that buys the most loss reduction per step *without* tipping into divergence — so it trains fastest. One cheap run answers a question a grid search pays for many times over.

2 Step 1 — The geometric schedule and the multiplier q

We want the learning rate to be *multiplied* by the same factor each step (geometric growth), because "good" learning rates span orders of magnitude and a log-spaced sweep covers them evenly. Write the learning rate at step i as

$$\text{lr}_i = \text{lr}_{\min} \cdot q^i, \quad i = 0, 1, \dots, \text{num_iter}$$

Two endpoints pin down q : at $i = 0$ we get $\text{lr}_{\min}$ (good), and at $i = \text{num_iter}$ we want exactly $\text{lr}_{\max}$. Setting $\text{lr}_{\min} q^{\text{num_iter}} = \text{lr}_{\max}$ and solving:

$$q^{\text{num_iter}} = \frac{\text{lr}_{\max}}{\text{lr}_{\min}} \quad \implies \quad q = \left(\frac{\text{lr}_{\max}}{\text{lr}_{\min}} \right)^{1/\text{num_iter}}$$

With our numbers $\text{lr}_{\max}/\text{lr}_{\min} = 1/10^{-5} = 10^5$, so

$$q = (10^5)^{1/100} = 10^{5/100} = 10^{0.05} = 1.122.$$

Steps per decade. Taking $\log_{10}$ of the schedule turns multiplication into a constant *additive* step in log-space:

$$\log_{10} \text{lr}_i = \log_{10} \text{lr}_{\min} + i \log_{10} q, \quad \log_{10} q = 0.05.$$

So each step adds 0.05 to the exponent, and a full decade (a factor of 10) takes $1/0.05 = 20$ steps. That is why the round powers of ten fall on round step numbers:

step i	$\log_{10} \text{lr}_i = -5 + 0.05 i$	$\text{lr}_i = 10^{-5} q^i$	landmark
0	-5.000	1.000×10^{-5}	start = $\text{lr}_{\min}$
20	-4.000	1.000×10^{-4}	one decade up
50	-2.500	3.162×10^{-3}	half-way (in log)
80	-1.000	1.000×10^{-1}	four decades up
100	0.000	1.000×10^0	end = $\text{lr}_{\max}$

Mini-example — this operation in isolation

The exponent is linear in i , so any landmark is one subtraction. Where does the learning rate first reach 3.162×10^{-3} (i.e. $10^{-2.5}$)? Solve $-5 + 0.05 i = -2.5 \Rightarrow i = 2.5/0.05 = 50$. And $q^{50} = 10^{0.05 \cdot 50} = 10^{2.5} = 316.228$, so $\text{lr}_{50} = 10^{-5} \cdot 316.228 = 3.162 \times 10^{-3}$. The “ $\div 0.05$ ” is just “20 steps per decade” in disguise.

3 Step 2 — The three regimes of the loss curve

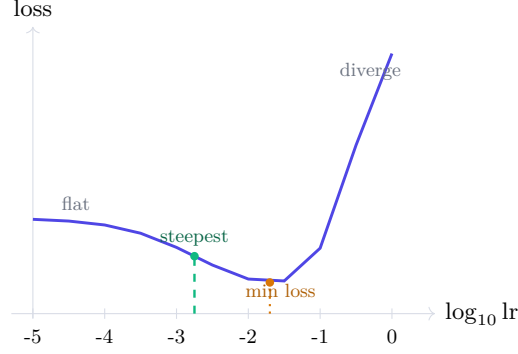
Plot the recorded loss ℓ_i against $\log_{10} \text{lr}_i$ and a canonical shape appears, in three regimes:

1. **Flat (lr too small).** The steps are so tiny the loss barely moves; the curve sits on its initial plateau near $\ln 3 \approx 1.099$.
2. **Descent (the useful band).** The learning rate is finally large enough to make real progress; the loss falls steeply. *This* is the band you want to train in.
3. **Divergence (lr too big).** Updates overshoot the minimum, the loss shoots back up and eventually explodes to NaN.

Tabulating the loss at the same landmark steps (from the reference program) makes the three regimes concrete:

step i	lr_i	loss ℓ_i	regime
0	1.000×10^{-5}	1.085	flat (too small)
20	1.000×10^{-4}	1.020	flat, starting to bend
50	3.162×10^{-3}	0.559	steep descent — useful band
80	1.000×10^{-1}	0.753	past the bottom, climbing (too big)
100	1.000×10^0	2.992	diverging

From $i = 0$ to 20 the loss drops only $1.085 \rightarrow 1.020$ (the flat plateau). From $i = 20$ to 50 it falls $1.020 \rightarrow 0.559$ — the descent. By $i = 80$ it has already turned around to 0.753, and by $i = 100$ it has diverged to 2.992, well above where it started.



Watch out

Do *not* pick the learning rate at the loss *minimum* (amber dot, $\text{lr} \approx 2.0 \times 10^{-2}$). That point is already on the edge of instability: it is the bottom only because the *next* step has not yet overshoot. Training there for real, batch after batch, tends to oscillate. Use a learning rate from the *descent*, where the curve is still falling fast — typically about one order of magnitude below the minimum-loss learning rate.

4 Step 3 — The steepest-descent pick: slope of the curve

”Steepest descent” means the learning rate at which the loss falls fastest *per unit of* $\log_{10} \text{lr}$ — the most negative slope of the curve. Approximate the slope by a finite difference between neighbouring landmark points:

$$\text{slope}_i \approx \frac{\ell_{i+\Delta} - \ell_{i-\Delta}}{\log_{10} \text{lr}_{i+\Delta} - \log_{10} \text{lr}_{i-\Delta}} = \frac{d\text{loss}}{d\log_{10} \text{lr}}.$$

Around the descent ($i = 40$ to 50 , i.e. lr from 10^{-3} to 3.162×10^{-3}):

$$\text{slope} \approx \frac{\ell_{50} - \ell_{40}}{\log_{10} \text{lr}_{50} - \log_{10} \text{lr}_{40}} = \frac{0.559 - 0.762}{(-2.5) - (-3.0)} = \frac{-0.203}{0.5} = -0.405.$$

Scanning the whole curve (Appendix A computes $\nabla \ell$ at every step and ignores the last few unstable points), the *most negative* slope is -0.413 , at step $i = 45$:

steepest descent at $i = 45$: $\text{lr}^* = 1.778 \times 10^{-3}$, $\ell = 0.661$, $\text{slope} = -0.413$

That $\text{lr}^* \approx 1.8 \times 10^{-3}$ is the suggested learning rate.

step i	lr_i	loss ℓ_i	slope $\frac{d\ell}{d\log_{10} \text{lr}}$	note
30	3.162×10^{-4}	0.925	-0.256	descent starting
40	1.000×10^{-3}	0.762	-0.387	steepening
45	1.778×10^{-3}	0.661	-0.413	steepest
50	3.162×10^{-3}	0.559	-0.394	still steep
60	1.000×10^{-2}	0.398	-0.222	flattening at bottom
70	3.162×10^{-2}	0.376	$+0.209$	past minimum, rising

5 Step 4 — The cross-check: minimum-loss over ten

A second, very common rule: find the learning rate at the *minimum* recorded loss, then back off by one order of magnitude (divide by 10). From the curve, the minimum loss is at step $i = 66$:

$$\min \text{ loss } \ell_{\min} = 0.359 \quad \text{at} \quad \text{lr}_{\min\text{-loss}} = 1.995 \times 10^{-2}.$$

Backing off by a decade:

$$\frac{\text{lr}_{\min\text{-loss}}}{10} = \frac{1.995 \times 10^{-2}}{10} = 1.995 \times 10^{-3}$$

Compare the two suggestions: steepest descent gives 1.778×10^{-3} , min-loss/10 gives 1.995×10^{-3} . They agree to within a factor of 1.1 — both say "about 2×10^{-3} ". That agreement is the whole point: the steepest-descent learning rate naturally sits roughly one decade below the minimum-loss learning rate, so the two rules converge on the same safe choice in the heart of the descent.

When this is the right choice

Reach for the range test whenever you face a *new* model or a *new* dataset and have no prior learning rate to copy. It is especially natural with one-cycle and cyclical schedules: the test directly hands you the schedule's *maximum* learning rate (the steepest-descent value, or the min-loss value as an upper bound), and you let the schedule anneal down from there. One sweep of a few hundred batches calibrates the most important hyper-parameter for the entire run.

6 Step 5 — The divergence guard and the perturbed weights

Stopping early. You do not need to finish all 100 steps once the loss clearly blows up. A standard guard halts the sweep as soon as the current loss exceeds $4\times$ the best loss seen so far:

$$\text{stop when } \ell_i > 4 \ell_{\text{best}}.$$

Here $\ell_{\text{best}} = 0.359$, so the threshold is $4 \times 0.359 = 1.437$. The loss first crosses it at step $i = 87$, where $\text{lr} = 2.239 \times 10^{-1}$ and $\ell = 1.518$. Everything past that point is noise and would only risk NaN — so the loop breaks there.

Resetting the model. Critically, the sweep *trained* the model for ~ 87 steps, including many with absurdly large learning rates near the end. Those updates have perturbed (often wrecked) the weights. Before real training you must restore the model to its pre-test state — the reference loop clones `state_dict()` up front and reloads it at the end.

Watch out

Two traps catch everyone once. (1) *Pick the descent, not the minimum.* The loss-minimum learning rate ($\approx 2.0 \times 10^{-2}$ here) is already near the edge; the steepest-descent value ($\approx 1.8 \times 10^{-3}$) trains fast *and* stably. (2) *Reset the weights after the test.* The range test is itself a (chaotic) training run; if you start your real run from the post-test weights, you begin from a damaged, large-learning-rate-corrupted state. Always reload the saved `state_dict()`.

7 The sweep, at a glance

#	Quantity	Value here	Computed as
1	multiplier q	1.122	$(\text{lr}_{\max}/\text{lr}_{\min})^{1/\text{num_iter}}$
2	steps per decade	20	$1/\log_{10} q$
3	lr at $i=0$	1.000×10^{-5}	$\text{lr}_{\min}$
4	lr at $i=50$	3.162×10^{-3}	$\text{lr}_{\min} q^{50}$
5	lr at $i=100$	1.000×10^0	$\text{lr}_{\max}$
6	steepest-descent lr	1.778×10^{-3}	arg min slope ($i=45$)
7	min-loss lr	1.995×10^{-2}	arg min loss ($i=66$)
8	min-loss lr / 10	1.995×10^{-3}	back off one decade
9	divergence guard lr	2.239×10^{-1}	first $\ell_i > 4\ell_{\text{best}}$ ($i=87$)
10	suggested lr	$\approx 2 \times 10^{-3}$	descent point (both rules agree)

The whole method is rows 1–2 (build the sweep), rows 3–5 (run it), rows 6–9 (read the curve), and row 10 (the answer). Then reset the weights.

8 Equation cheat-sheet

Geometric schedule	$\text{lr}_i = \text{lr}_{\min} q^i$
Multiplier	$q = (\text{lr}_{\max}/\text{lr}_{\min})^{1/\text{num_iter}}$
Log-space step	$\log_{10} \text{lr}_i = \log_{10} \text{lr}_{\min} + i \log_{10} q$
Steps per decade	$1/\log_{10} q$
Slope of curve	$\text{slope}_i = \frac{d \text{loss}}{d \log_{10} \text{lr}}$
Steepest-descent pick	$\text{lr}^* = \text{lr}_{\arg \min_i \text{slope}_i}$
Min-loss / 10 rule	$\text{lr}_{\min \text{-loss}}/10$
Divergence guard	stop when $\ell_i > 4\ell_{\text{best}}$
After the test	reload saved <code>state_dict()</code>

Appendix A Reference implementation

Every number above is reproducible from the following program. The first block is the PyTorch range-test loop (exponential learning rate, recorded loss, weight reset); the second block reproduces the exact learning rates, the synthetic-but-smooth loss curve, and both suggestion rules used in the text.

```
import torch, torch.nn as nn, numpy as np

def lr_finder(model, X, y, criterion,
              lr_min=1e-5, lr_max=1.0, num_iter=100):
    # save state so we can undo the perturbation the test causes
    orig_state = {k: v.clone() for k, v in model.state_dict().items()}
    opt = torch.optim.SGD(model.parameters(), lr=lr_min)
    q = (lr_max / lr_min) ** (1.0 / num_iter)  # = 1.122
    lr = lr_min
    lrs, losses, best = [], [], float('inf')
    for i in range(num_iter):
        opt.zero_grad()
        loss = criterion(model(X), y)
        loss.backward()
        opt.step()
        lrs.append(lr); losses.append(loss.item())
        best = min(best, loss.item())
        if loss.item() > 4 * best:  # divergence guard
            break
        lr *= q  # exponential growth
        for pg in opt.param_groups:
            pg['lr'] = lr
        model.load_state_dict(orig_state)  # reset perturbed weights
    return np.array(lrs), np.array(losses)

# ---- the exact geometry used in the document ----
lr_min, lr_max, num_iter = 1e-5, 1.0, 100
q = (lr_max / lr_min) ** (1.0 / num_iter)
print("q =", round(q, 3))  # 1.122
print("steps/decade =", round(1/np.log10(q)))  # 20
i = np.arange(num_iter + 1)
lr = lr_min * q**i
for k in [0, 20, 50, 80, 100]:
    print(k, f"{lr[k]:.3e}")  # 1.000e-05 1.000e-04 3.162e-03 1.000e-01 1.000e+00

# ---- a smooth loss curve with the three regimes ----
sig = lambda z: 1/(1+np.exp(-z))
lg = np.log10(lr)
loss = 1.099 - 0.92*sig((lg+2.7)/0.55) + 3.2*sig((lg+0.55)/0.28)
slope = np.gradient(loss, lg)

best_idx = int(np.argmin(slope[:-5]))  # steepest descent (ignore tail)
min_idx = int(np.argmin(loss))  # minimum loss
print("steepest:", best_idx, f"{lr[best_idx]:.3e}", round(loss[best_idx],3),
      round(slope[best_idx],3))  # 45 1.778e-03 0.661 -0.413
print("min loss:", min_idx, f"{lr[min_idx]:.3e}", round(loss[min_idx],3))  # 66 1.995e-02 0.359
print("min-loss/10:", f"{lr[min_idx]/10:.3e}")  # 1.995e-03

best = loss.min()
explode = np.where(loss > 4*best)[0][0]  # divergence guard
print("guard:", explode, f"{lr[explode]:.3e}", round(loss[explode],3))  # 87 2.239e-01 1.518
```

— end of document —